Intern Task Brief: Medical PDF Question-Answering System (RAG)

Duration: 7 working days | **Level:** Intern | **Cost:** Free tools only

1. Overview

Build a Retrieval-Augmented Generation (RAG) system that answers questions from printable medical pages (PDFs such as guidelines, drug leaflets and patient handouts). Every answer must cite its source page, and the whole system must be measured with quantitative metrics and statistical analysis.

2. What You Will Learn

- Extracting and cleaning text from PDFs
- Chunking, embeddings and vector databases
- Building a RAG pipeline with a free LLM
- Evaluating RAG with retrieval, generation and safety metrics
- Basic statistics: confidence intervals, significance tests, effect size

3. Scope

In scope

- 20 to 50 open-source medical PDFs
- Retrieval, answer generation with citations, evaluation, and a short report
- A simple CLI or demo UI

4. Tech Stack (All Free)

Component	Tool
PDF loading	PyMuPDF or pdfplumber (Tesseract OCR for scanned pages)
Chunking	explore chunking approch

Component	Tool
Embedding model	Hugging Face `BAAI/bge-small-en-v1.5` (alternatives: `bge-base`, `NeuML/pubmedbert-base-embeddings`)
Vector database	Your choice from ChromaDB, FAISS or Qdrant local
LLM	Gemini free tier, Cohere trial key, or Hugging Face Inference API
Evaluation	see (section 6)
UI (optional)	Streamlit or Gradio

5. Work Plan

Day 1 to 2: Data and Ingestion

- [ ] Collect 20 to 50 PDFs
- [ ] Record metadata for each file: title, source URL, publication date
- [ ] Extract text page by page
- [ ] Implement 2 to 4 chunking strategies and build chunks for each; pick the final one from the results
- [ ] Attach metadata to each chunk: `doc_id`, `page`, `section`

Day 3: Embeddings and Vector Database

- [ ] Embed all chunks with the BGE model
- [ ] Use the query prefix `Represent this sentence for searching relevant passages:` for queries only, not for documents
- [ ] Normalize vectors and store them with metadata in each vector database you are comparing
- [ ] Write a script that rebuilds the index from scratch

Day 4: RAG Pipeline

- [ ] Retrieve top-k chunks (k is a setting; fix it from the accuracy sweep in section 7)
- [ ] implement query rewrite

- [ ] Optionally rerank the results
- [ ] Send question and context to the LLM
- [ ] Follow these prompt rules:
  - Answer only from the provided context
  - Cite sources as `[doc, page]`
  - Reply "I don't know" if the context does not contain the answer
- [ ] Day 6 to 7: Metrics, Experiments and Report
- [ ] Compute all metrics in section 6
- [ ] Compare at least 2 configurations (section 7)
- [ ] Write the final report

6. Evaluation Metrics

6.1 Retrieval

- Hit Rate@k
- Recall@k
- Precision@k
- MRR (Mean Reciprocal Rank)
- nDCG@k

7. Experiments

Compare at least 2 configurations. Suggested variables:

Variable	Values to try
Chunk size	Strategy and size 300, 600, 900 tokens
Top-k	1, 3, 5, 8, 10, 15 (choose by accuracy, see 4.1)
Reranker	With and without
LLM	Two different free models

Change **one variable at a time** and keep everything else fixed.

8. Deliverables

#	Deliverable	Format
1	Source code, `requirements.txt`, `.env.example`	Git repository
5	Short report (architecture diagram, experiment table, key findings, limitations)	PDF or Markdown
6	Demo UI or CLI	Streamlit, Gradio or CLI